

Ascon AEAD128 RTL Reading Guide

A short module-by-module guide to the VHDL implementation

Purpose. I wrote this guide to explain my RTL and its self-checking VHDL testbench. I cover the shared definitions, permutation engine, AEAD controller, Nexys A7 demonstration wrapper, and testbench in that order. My implementation is an unmasked functional baseline. I did not add side-channel or fault-hardening features.

1 Overall Design

I implement NIST Ascon-AEAD128. A transaction accepts a command with a key and nonce, consumes associated-data beats and payload beats, then either emits a tag during encryption or checks an input tag during decryption. I store the shared 320-bit state as five 64-bit words, x0 through x4. The lower 128 bits, x0 and x1, are the rate; x2 through x4 are capacity.

My controller owns the protocol and transaction state. It starts the iterative permutation unit whenever Ascon needs p12 or p8. I run one round per clock. I use p12 during initialization and finalization, and p8 after associated-data blocks and full payload blocks.

Read the transaction in this order

Step	What happens in RTL
1. Command	In ST_IDLE, I accept cmd_valid_i and load the IV, key, nonce, and encryption/decryption mode.
2. Initialize	I request p12, then XOR the key into state words x3 and x4.
3. Associated data	I mask each accepted AD beat into the rate. I run p8 after each non-empty block and apply domain separation before payload.
4. Payload	The rate XOR produces ciphertext or tentative plaintext. I run p8 after full blocks; I pad a final partial block and move directly to finalization.
5. Finalize	I XOR the key into x2 and x3, run p12, and form the 128-bit tag from x3/x4 XOR key.
6. Finish safely	For encryption, I wait for tag acceptance. For decryption, I compare the tag and pulse commit_o or discard_o. I zeroize storage before done_o.

Handshake rule. I transfer a stream beat on a rising edge when valid and ready are both 1. My core exposes ready only in the expected phase. A 128-bit beat carries up to 16 bytes. keep bit 0 maps to data bits 7 downto 0, and valid keep bits must be contiguous from lane 0.

2 ascon_pkg.vhd Shared Types and Functions

I use this package for shared types, constants, and combinational helper functions. It is not a clocked hardware block. Read it first because I define the state layout and byte handling here.

Definitions and functions

Item	Purpose
word64_t, rate_t, state_t	64-bit word, 128-bit rate, and 320-bit full state types.
ASCON_AEAD128_IV_C	Final NIST AEAD128 initialization value placed in x0.
ASCON_DSEP_C	Domain-separation bit XORed into x4 after associated data.

Item	Purpose
state_word / set_state_word	Read or replace one 64-bit word in a 320-bit state. Index 0 occupies bits 63 down to 0.
state_rate / set_state_rate	Read or replace the lower 128-bit rate portion of the state.
round_constant	Computes one byte from the F0 through 4B round-constant sequence.
ascon_round	Applies one constant addition, substitution layer, and diffusion layer to all five words.
keep_is_contiguous / keep_count	Validate a final-beat byte mask and count its valid byte lanes.
keep_mask	Expands each keep bit into its matching 8-bit all-ones or all-zeros mask.
padding_bit	Returns a 128-bit value with byte 01 immediately after the valid input bytes.

Code path. My controller calls the state and byte helpers while it absorbs input. My permutation engine calls ascon_round once per active clock. The package has no ports because I call its functions directly from other VHDL units.

3 ascon_permutation.vhd Iterative Permutation

I use this module as my reusable p8/p12 datapath. It has one 320-bit state register and calls the package's combinational ascon_round function once per clock. I reuse one round instead of unrolling all rounds to use less area.

Ports

Port	Dir	Meaning
clk_i	in	Clock for the state register and round counter.
rst_i	in	Active-high synchronous reset; clears state and control.
start_i	in	One-cycle request accepted only while the engine is idle.
rounds_i[3:0]	in	Requested round count. The controller uses 12 or 8.
state_i[319:0]	in	Initial state captured when start_i is accepted.
busy_o	out	High while rounds are running.
done_o	out	One-cycle pulse after the final round is registered.
state_o[319:0]	out	Current state register; final result when done_o is high.

Code path

When the engine is idle and start_i is 1, I capture state_i, choose the first round as 12 minus rounds_i, and raise busy. On each later rising edge, I compute ascon_round(state_reg, round_number_reg) and write the result back. After round 11, I drop busy and pulse done. Therefore p12 uses indices 0 through 11 and p8 uses indices 4 through 11.

4 ascon_aead128_core.vhd Streaming AEAD Controller

This is my main reusable RTL block. Its finite-state machine stores transaction state, manages ready/valid I/O, starts the permutation, and enforces my zeroization and authenticated-decryption rules.

Ports

Group	Ports	Meaning
Clock	clk_i, rst_i	Synchronous clock and active-high reset.
Command input	cmd_valid_i, cmd_ready_o, cmd_decrypt_i, key_i, nonce_i	Start only when ready. decrypt 0 selects encryption; 1 selects decryption. Key and nonce are each 128 bits.
Data input	in_valid_i, in_ready_o, in_kind_i, in_data_i, in_keep_i, in_last_i	Associated data when kind is 0; payload when kind is 1. Data has up to 16 valid byte lanes.
Decrypt tag input	tag_in_valid_i, tag_in_ready_o, tag_in_i	Supplies the expected 128-bit tag after payload processing.
Payload output	out_valid_o, out_ready_i, out_data_o, out_keep_o, out_last_o	Ciphertext during encryption or tentative plaintext during decryption.
Encrypt tag output	tag_out_valid_o, tag_out_ready_i, tag_out_o	Generated 128-bit tag held valid until accepted.
Status	busy_o, done_o, error_o	Busy is level-based; done and error are one-clock status events.
Authentication	auth_valid_o, auth_ok_o, commit_o, discard_o	Decryption result. External logic must release plaintext only on commit_o and discard it on discard_o.

Key local functions

I use xor_key_after_initialization to XOR key halves into x3/x4. xor_key_before_finalization XORs key halves into x2/x3. tag_from_final_state forms the tag from x3/x4 XOR key. I keep these operations in small functions instead of repeating the key placement inside the FSM cases.

Code path through the FSM

States	Action
ST_IDLE -> ST_INIT_START -> ST_INIT_WAIT	Accept command, load IV/key/nonce state, request p12, then apply post-initialization key XOR.
ST_AD_WAIT and AD permutation states	Accept only kind=0. Validate keep/last, absorb a masked AD block, run p8 as required, then apply domain separation.
ST_MSG_WAIT -> ST_MSG_OUTPUT	Accept only kind=1. Encrypt by XORing plaintext into the rate; decrypt by XORing rate with ciphertext while storing ciphertext into rate. Present a payload beat and wait for out_ready_i.
ST_MSG_PERM states	Run p8 after a full payload block. A final full block receives an extra empty padding block before finalization.
ST_FINAL states	XOR key into x2/x3, run p12, and compute/store the tag.
ST_TAG_OUT or ST_TAG_IN	For encryption, hold tag until accepted. For decryption, compare tag and pulse authentication plus commit or discard.
ST_ZEROIZE -> ST_DONE	Clear state, key, mode, phase flags, payload buffer, tag, and authentication storage. Then pulse done and return idle.

Error path. If I receive a wrong phase, non-contiguous keep mask, or non-final partial beat, I pulse error_o and enter ST_ZEROIZE. In decryption mode, I also pulse discard_o. I use the same cleanup path after normal tag handling.

5 ascon_demo_top.vhd Nexys A7 Demonstration Wrapper

I wrote this top-level module for a board demonstration, not as the general streaming interface. I instantiate ascon_aead128_core, provide four fixed known-answer test cases, and show the result through switches, buttons, and LEDs.

Ports

Port	Dir	Meaning
CLK100MHZ	in	Nexys A7 100 MHz clock, connected to the core clock.
SW[9:0]	in	Selects test vector, mode, display source, byte lane, and payload block.
BTNC	in	Start button. It is synchronized through two flip-flops and edge detected.
BTNU	in	Synchronous reset, connected to the core reset.
LED[12:0]	out	Eight selected result bits plus busy, done, pass, fail, and error indicators.

Code path

In UI_IDLE, I capture the chosen switches on one clean BTNC edge. UI_COMMAND sends the core command. UI_AD0/UI_AD1 drive fixed associated-data beats; UI_MSG0/UI_MSG1 drive plaintext for encryption or ciphertext for decryption. UI_WAIT_PAYLOAD waits for either a tag input opportunity or generated tag. I capture payload and tag outputs, compare them with the fixed expected values, wait for core_done, and latch pass or fail. A combinational multiplexer selects one result byte for LED[7:0].

Reading note. I permanently drive out_ready_i and tag_out_ready_i high in this wrapper, so it does not demonstrate output backpressure. Use the core interface when connecting a processor or DMA engine.

6 tb_ascon_aead128.vhd Self Checking Testbench

I wrote this as simulation-only VHDL, so it is not synthesizable FPGA hardware. I instantiate my AEAD core as the DUT, read official known-answer vectors, drive the same ready/valid interface used by hardware, check results in the simulator, and write result logs.

External interface

Item	Meaning
Entity ports	None. A testbench is the top of the simulation hierarchy and creates its own clock, reset, and DUT signals.
G_VECTOR_FILE generic	Input path for the compact known-answer vector file. Default: verification/vectors/xsim_vectors.txt.
Internal signals	clk/rst plus one-to-one replicas of every core port, including command, input stream, output stream, tag, and status signals.
Output files	simulation_results.txt is human-readable. simulation_vectors.txt is pipe-delimited for scripts that calculate latency and throughput.

Key procedures and code path

Procedure or block	What it does
Clock and cycle counter	Generates a 10 ns clock and counts simulation cycles independently of DUT reset.
wait_ready	Waits until a ready or valid signal becomes 1 before the caller continues.
start_command	Waits for cmd_ready, applies mode/key/nonce, pulses cmd_valid, and records the starting cycle.
send_beat / send_ad	Drives a correctly formed stream beat. send_ad creates an empty, one-block, or two-block associated-data phase.
send_payload	Sends plaintext or ciphertext beats, waits for output valid, captures output data, and pulses out_ready.
run_vector	Runs one vector twice: encryption checks ciphertext/tag; decryption checks recovered plaintext and successful authentication/commit.

Procedure or block	What it does
Main stimulus	Opens the vector file, resets the DUT, parses each record, calls run_vector, writes summaries, then asserts 1,089 vectors and 2,178 passing directional checks.

What I check. This bench checks expected ciphertext, plaintext, tag, error status, authentication result, and completion behavior against the supplied official vector records. I also use it to write per-case latency logs. It does not establish physical-board behavior or resistance to side-channel and fault attacks.

7 Suggested Reading Order

1. Read ascon_pkg.vhd for my state layout, little-endian byte lanes, masks, padding, and one round. 2. Read ascon_permutation.vhd to see how I run one round per clock. 3. Read ascon_aead128_core.vhd from ST_IDLE through ST_DONE while tracing one encryption transaction. 4. Trace decryption again and focus on ST_TAG_IN, commit_o, discard_o, and ST_ZEROIZE. 5. Read ascon_demo_top.vhd as my board-level driver. 6. Read tb_ascon_aead128.vhd to see how I exercise and check the core in simulation.

Integration rule. My decrypted payload is not authenticated when it first appears at out_data_o. Store it in an external quarantine buffer and release it only if commit_o pulses after a successful tag comparison.